

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
УЛЬЯНОВСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

Факультет информационных систем и технологий
Кафедра «Измерительно-вычислительные комплексы»

**Отчет по лабораторной работе №4
по дисциплине «Системы искусственного интеллекта»**

Выполнил:
студент группы ИСТбд-31 Коняхин Н.С

Проверил:
Преподаватель Шишкин В.В.

Ульяновск, 2026

Цель:

Разработать алгоритм ExtraTreeClassifier в самописном варианте и сравнить с работой модели из библиотеки SCikit-learn. Рассмотреть работу на разном наборе данных

1. Выбор датасета

Мой выбор пал на стандартный датасет из библиотеки scikit-learn «iris». Он прост для начинающих и хорошо структурирован.

2. Выбор количества признаков

оптимальным количеством является $\sqrt{n}$ из числа признаков датасета. В таблице 1 приведена сравнительная характеристика иных вариантов количества признаков

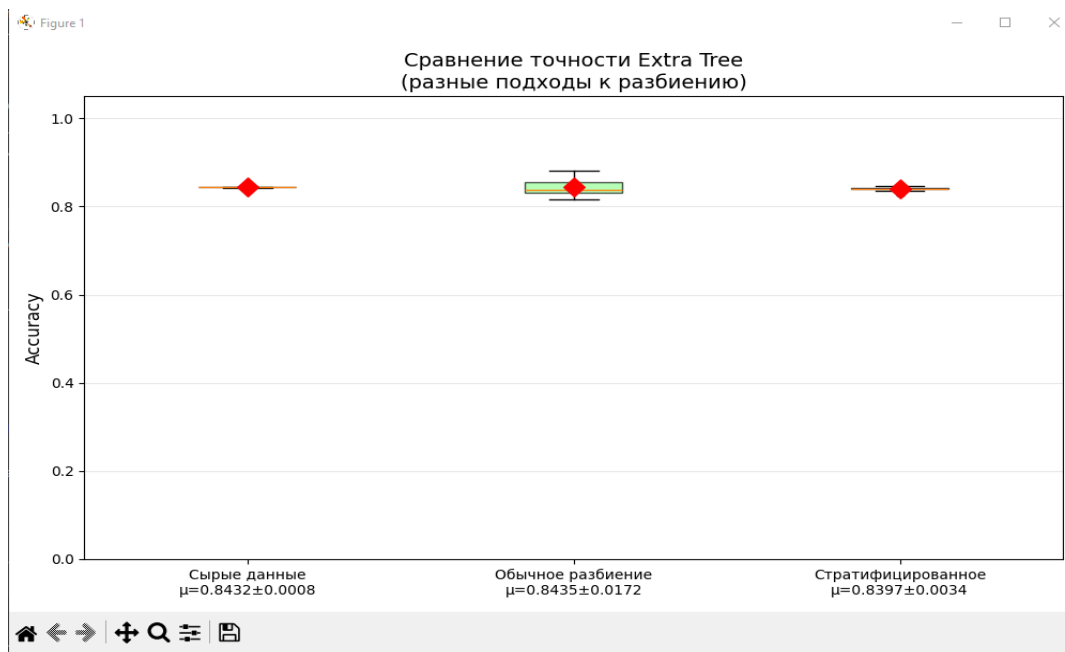
Количество признаков	Разнообразие деревьев	Сила каждого дерева	Итоговое качество
1	Очень высокое	Очень низкое	Среднее (нужно много деревьев)
$\sqrt{n}$	Высокое	Среднее	Лучшее
$n/2$	Среднее	Высокое	Хорошее
n (все)	Низкое (деревья похожи)	Очень высокое	Среднее (переобучение)

Таблица 1

Данное количество предложил Лео Брейман в статье «Random Forests»

3. Обучение модели на самописной реализации алгоритма

Обучим модель на сырых данных ,затем на подготовленных используя два метода формирования тестового и тренировочного набора. Первый это «обычное разбиение» ,второй «стратифицированное».



Проанализировав данный график можно сделать вывод. Что на сырых данных модель переобучилась и выдала нереалистично высокую точность. Обычное распределение дает более точную оценку ,но обладает высокой дисперсией . Стратифицированное наиболее подходящий вариант подготовки данных так как он дает более «чистый» результат и низкую дисперсию.Ниже представлена таблица сравнения методов подготовки данных

4. Использование библиотеки SCIKIT-learn

Повторим ранее проведенный эксперимент с реализацией

Метод	Среднее	Стд. откл.	Min	Max
Сырые данные (обучение+тест)	0.8410 ± 0.0009	0.8395	0.8433	
Обычное разбиение (80/20)	0.8419 ± 0.0195	0.7994	0.8809	
Стратифицированное разбиение (80/20)	0.8401 ± 0.0016	0.8339	0.8433	

Из данных графиков можно заметить что результат оказался одинаковым. И все методы выдали точные такие же результаты что и самописной вариацией.

Проверка на тестовом датасете

однаков на тестовом датасете точность резко снижается.Можно предположить это основанно на дисбалансе классов в обучающем датасете и слабых сигнала во всех признаках кроме X6.

```
Оценка точности:
raw          | Accuracy: 0.4854 | Верно: 166/342
random       | Accuracy: 0.4854 | Верно: 166/342
stratified   | Accuracy: 0.4854 | Верно: 166/342
```

Вывод:

Выполнив данную работу по обучению модели на алгоритме extraTreeClassifier в двух реализациях используя несколько методов подготовки данных. Можно сделать вывод о необходимости подготавливать перед обучением модели ,что избежать ее переобучения и повысить ее точность.

```
import sys
from typing import List, Tuple, Any
import numpy as np
import random
```

```

import math
from collections import Counter

#
=====
#
# РЕАЛИЗАЦИЯ EXTRA TREE ALGORITHM (ВАШ АЛГОРИТМ)
#
=====
#

class Node:
    """Узел дерева решений"""
    __slots__ = ['feature_idx', 'threshold', 'left', 'right', 'prediction']

    def __init__(self, feature_idx=None, threshold=None, left=None, right=None,
prediction=None):
        self.feature_idx = feature_idx
        self.threshold = threshold
        self.left = left
        self.right = right
        self.prediction = prediction

class ExtraTree:
    """Одно дерево Extra Tree"""

    def __init__(self, max_depth=5, min_samples_split=2, max_features=None,
        n_random_splits=10, random_state=None):
        self.max_depth = max_depth
        self.min_samples_split = min_samples_split
        self.max_features = max_features
        self.n_random_splits = n_random_splits
        self.random_state = random_state
        self.rng = np.random.RandomState(random_state) if random_state else
np.random.RandomState()
        self.root = None

    def _gini(self, y):
        """Расчёт индекса Джини"""
        if len(y) == 0:
            return 0.0
        # Используем int для индексов
        y_int = y.astype(int) if hasattr(y, 'astype') else np.array(y, dtype=int)
        max_label = int(np.max(y_int)) + 1
        counts = np.bincount(y_int, minlength=max_label)
        probs = counts / len(y)
        return 1.0 - np.sum(probs ** 2)

    def _get_n_features(self, n_features):
        """Определяет количество признаков для рассмотрения"""
        if self.max_features is None:
            return n_features
        elif isinstance(self.max_features, str):
            if self.max_features == 'sqrt':
                return max(1, int(np.sqrt(n_features)))
            elif self.max_features == 'log2':
                return max(1, int(np.log2(n_features)))
            else:
                return n_features
        elif isinstance(self.max_features, float):

```

```

        return max(1, int(self.max_features * n_features))
    elif isinstance(self.max_features, int):
        return min(self.max_features, n_features)
    else:
        return n_features

def _best_split(self, X, y):
    """Поиск лучшего случайного разделения"""
    n_samples, n_features = X.shape

    if n_samples < self.min_samples_split:
        return None

    # Определяем количество признаков для рассмотрения
    n_feats = self._get_n_features(n_features)

    # Случайный выбор признаков
    feature_indices = self.rng.choice(n_features, size=n_feats, replace=False)

    best_gini = float('inf')
    best_feat = None
    best_thr = None

    for feat_idx in feature_indices:
        col = X[:, feat_idx].copy()
        min_val = col.min()
        max_val = col.max()

        if min_val == max_val:
            continue

        # Генерация случайных порогов
        thresholds = self.rng.uniform(min_val, max_val, size=self.n_random_splits)

        for thr in thresholds:
            left_mask = col <= thr
            right_mask = ~left_mask

            left_size = left_mask.sum()
            right_size = right_mask.sum()

            if left_size == 0 or right_size == 0:
                continue

            left_gini = self._gini(y[left_mask])
            right_gini = self._gini(y[right_mask])

            gini = (left_size / n_samples) * left_gini + (right_size / n_samples) * right_gini

            if gini < best_gini:
                best_gini = gini
                best_feat = feat_idx
                best_thr = thr

    return (best_feat, best_thr) if best_feat is not None else None

def _build_tree(self, X, y, depth):
    """Рекурсивное построение дерева"""
    # Преобразуем y в плоский массив int
    y = np.asarray(y).flatten()

    unique_classes = np.unique(y)

```

```

# Условия остановки
if len(unique_classes) == 1:
    return Node(prediction=int(unique_classes[0]))

if depth >= self.max_depth:
    y_int = y.astype(int)
    max_label = int(np.max(y_int)) + 1
    most_common = np.bincount(y_int, minlength=max_label).argmax()
    return Node(prediction=int(most_common))

if len(y) < self.min_samples_split:
    y_int = y.astype(int)
    max_label = int(np.max(y_int)) + 1
    most_common = np.bincount(y_int, minlength=max_label).argmax()
    return Node(prediction=int(most_common))

split = self._best_split(X, y)
if split is None:
    y_int = y.astype(int)
    max_label = int(np.max(y_int)) + 1
    most_common = np.bincount(y_int, minlength=max_label).argmax()
    return Node(prediction=int(most_common))

feat_idx, threshold = split

left_mask = X[:, feat_idx] <= threshold
right_mask = ~left_mask

left_child = self._build_tree(X[left_mask], y[left_mask], depth + 1)
right_child = self._build_tree(X[right_mask], y[right_mask], depth + 1)

return Node(feature_idx=feat_idx, threshold=threshold,
            left=left_child, right=right_child)

def fit(self, X, y):
    """Обучение дерева"""
    # Преобразуем входные данные в numpy массивы
    X = np.asarray(X)
    y = np.asarray(y).flatten()
    self.root = self._build_tree(X, y, depth=0)
    return self

def _predict_one(self, x, node):
    """Предсказание для одного объекта"""
    if node.prediction is not None:
        return node.prediction
    if x[node.feature_idx] <= node.threshold:
        return self._predict_one(x, node.left)
    return self._predict_one(x, node.right)

def predict(self, X):
    """Предсказание для массива объектов"""
    X = np.asarray(X)
    return np.array([self._predict_one(x, self.root) for x in X])

```

```

class ExtraTreeClassifier:

```

```

    """Ансамбль Extra Trees"""

```

```

    def __init__(self, n_estimators=10, max_depth=5, min_samples_split=2,
                  max_features='sqrt', n_random_splits=10, random_state=None):

```

```

self.n_estimators = n_estimators
self.max_depth = max_depth
self.min_samples_split = min_samples_split
self.max_features = max_features
self.n_random_splits = n_random_splits
self.random_state = random_state
self.rng = np.random.RandomState(random_state) if random_state else
np.random.RandomState()
self.trees = []

def fit(self, X, y):
    """Обучение ансамбля"""
    X = np.asarray(X)
    y = np.asarray(y).flatten()

    self.trees = []
    for i in range(self.n_estimators):
        tree_seed = self.random_state + i if self.random_state is not None else None
        tree = ExtraTree(
            max_depth=self.max_depth,
            min_samples_split=self.min_samples_split,
            max_features=self.max_features,
            n_random_splits=self.n_random_splits,
            random_state=tree_seed
        )
        tree.fit(X, y)
        self.trees.append(tree)
    return self

def predict(self, X):
    """Предсказание с голосованием"""
    X = np.asarray(X)
    all_preds = np.array([tree.predict(X) for tree in self.trees])
    predictions = []
    for i in range(X.shape[0]):
        votes = all_preds[:, i]
        max_label = int(np.max(votes)) + 1 if len(votes) > 0 else 2
        most_common = np.bincount(votes.astype(int), minlength=max_label).argmax()
        predictions.append(int(most_common))
    return np.array(predictions)

def score(self, X, y):
    """Точность модели"""
    X = np.asarray(X)
    y = np.asarray(y).flatten()
    pred = self.predict(X)
    return np.mean(pred == y)

#
=====
#
# ФУНКЦИИ ДЛЯ РАЗБИЕНИЯ ДАННЫХ (ТОЛЬКО СТАНДАРТНАЯ БИБЛИОТЕКА)
#
=====
#

def train_test_split_manual(X, y, test_size=0.2, random_state=None):
    """
    Ручная реализация train_test_split без sklearn
    """
    if random_state is not None:

```

```

    random.seed(random_state)
    np.random.seed(random_state)

    n_samples = X.shape[0]
    indices = list(range(n_samples))
    random.shuffle(indices)

    n_test = int(n_samples * test_size)
    test_indices = indices[:n_test]
    train_indices = indices[n_test:]

    X_train = X[train_indices]
    y_train = y[train_indices]
    X_test = X[test_indices]
    y_test = y[test_indices]

    return X_train, X_test, y_train, y_test

def stratified_shuffle_split_manual(X, y, test_size=0.2, random_state=None):
    """
    Ручная реализация стратифицированного разбиения без sklearn
    Сохраняет пропорции классов в train и test
    """
    if random_state is not None:
        random.seed(random_state)
        np.random.seed(random_state)

    n_samples = X.shape[0]
    n_test = int(n_samples * test_size)

    # Получаем уникальные классы и их индексы
    y = np.asarray(y).flatten()
    unique_classes = np.unique(y)
    train_indices = []
    test_indices = []

    for cls in unique_classes:
        # Индексы объектов данного класса
        cls_indices = np.where(y == cls)[0].tolist()
        random.shuffle(cls_indices)

        # Сколько объектов этого класса должно попасть в тест
        n_cls = len(cls_indices)
        n_cls_test = int(n_cls * test_size)

        # Добавляем в test и train
        test_indices.extend(cls_indices[:n_cls_test])
        train_indices.extend(cls_indices[n_cls_test:])

    # Перемешиваем финальные списки
    random.shuffle(train_indices)
    random.shuffle(test_indices)

    X_train = X[train_indices]
    y_train = y[train_indices]
    X_test = X[test_indices]
    y_test = y[test_indices]

    return X_train, X_test, y_train, y_test

```



```

#
=====
#
# ОСНОВНОЙ ЭКСПЕРИМЕНТ
#
=====
#

def run_experiment(X, y, n_runs=30, n_estimators=20, max_depth=5):
    """
    Запуск эксперимента с тремя вариациями обучения

    Возвращает словарь с результатами ассигасу для каждого подхода
    """
    results = {
        'raw_data': [],
        'random_split': [],
        'stratified_split': []
    }

    for run in range(n_runs):
        print(f" Итерация {run + 1}/{n_runs}", end='\r')
        sys.stdout.flush()

        # ===== 1. Сырые данные (обучение и тест на всех данных) =====
        model_raw = ExtraTreeClassifier(
            n_estimators=n_estimators,
            max_depth=max_depth,
            max_features='sqrt',
            n_random_splits=10,
            random_state=run
        )
        model_raw.fit(X, y)
        accuracy_raw = model_raw.score(X, y)
        results['raw_data'].append(accuracy_raw)

        # ===== 2. Метод А: Обычное случайное разбиение =====
        X_train, X_test, y_train, y_test = train_test_split_manual(
            X, y, test_size=0.2, random_state=run
        )
        model_random = ExtraTreeClassifier(
            n_estimators=n_estimators,
            max_depth=max_depth,
            max_features='sqrt',
            n_random_splits=10,
            random_state=run
        )
        model_random.fit(X_train, y_train)
        accuracy_random = model_random.score(X_test, y_test)
        results['random_split'].append(accuracy_random)

        # ===== 3. Метод В: Стратифицированное разбиение =====
        X_train_strat, X_test_strat, y_train_strat, y_test_strat = stratified_shuffle_split_manual(
            X, y, test_size=0.2, random_state=run
        )
        model_strat = ExtraTreeClassifier(
            n_estimators=n_estimators,
            max_depth=max_depth,
            max_features='sqrt',
            n_random_splits=10,
            random_state=run
        )

```

```

model_strat.fit(X_train_strat, y_train_strat)
accuracy_strat = model_strat.score(X_test_strat, y_test_strat)
results['stratified_split'].append(accuracy_strat)

print() # переводим строку после прогресс-бара
return results

def calculate_statistics(results):
    """Вычисляет среднее и стандартное отклонение для каждого подхода"""
    stats = {}
    for key, values in results.items():
        stats[key] = {
            'mean': np.mean(values),
            'std': np.std(values),
            'min': np.min(values),
            'max': np.max(values)
        }
    return stats

def plot_results(results, stats):
    """Построение boxplot с результатами"""
    try:
        import matplotlib.pyplot as plt

        fig, ax = plt.subplots(figsize=(10, 6))

        # Подготовка данных для boxplot
        data_to_plot = [
            results['raw_data'],
            results['random_split'],
            results['stratified_split']
        ]

        labels = [
            f'Сырые данные\n(обучение+тест)\nμ={stats["raw_data"]["mean"]:.4f} ± {stats["raw_data"]["std"]:.4f}',
            f'Обычное разбиение\n(80/20)\nμ={stats["random_split"]["mean"]:.4f} ± {stats["random_split"]["std"]:.4f}',
            f'Стратифицированное\nразбиение\nμ={stats["stratified_split"]["mean"]:.4f} ± {stats["stratified_split"]["std"]:.4f}'
        ]

        # Создаём boxplot
        bp = ax.boxplot(data_to_plot, labels=labels, patch_artist=True)

        # Настройка цветов
        colors = ['#FF9999', '#99FF99', '#9999FF']
        for patch, color in zip(bp['boxes'], colors):
            patch.set_facecolor(color)
            patch.set_alpha(0.7)

        # Добавляем средние значения точками
        for i, (key, data) in enumerate(results.items(), 1):
            means = stats[key]['mean']
            ax.scatter(i, means, color='red', s=100, zorder=3,
                       marker='D', label='Среднее' if i == 1 else "")

        ax.set_ylabel('Accuracy (Точность)', fontsize=12)
        ax.set_title('Сравнение точности Extra Tree при разных подходах к обучению\n'
                     f'({len(results["raw_data"])} запусков)',

```

```

        fontsize=14)
ax.set_ylim(0, 1.05)
ax.grid(axis='y', alpha=0.3)

# Добавляем легенду
ax.legend(loc='lower right')

# Добавляем аннотацию с интерпретацией
ax.text(0.02, 0.02,
        'Интерпретация:\n'
        '• Сырые данные завышают оценку (переобучение)\n'
        '• Разбиение даёт более честную оценку\n'
        '• Стратификация снижает дисперсию',
        transform=ax.transAxes,
        fontsize=9,
        verticalalignment='bottom',
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

plt.tight_layout()
plt.show()

except ImportError:
    print("\n⚠ Matplotlib не установлен. Пропускаем построение графика.")
    print("Установите matplotlib: pip install matplotlib")

#
=====
# ЗАПУСК ЭКСПЕРИМЕНТА
#
=====

def main():
    print("=" * 70)
    print("ЭКСПЕРИМЕНТ: Сравнение подходов к обучению Extra Tree")
    print("=" * 70)

    # Загрузка датасета Iris
    print("\n1. Загрузка датасета Iris...")
    from sklearn.datasets import load_iris
    iris = load_iris()
    X = iris.data
    y = iris.target
    print(f"   □ Загружено: {X.shape[0]} объектов, {X.shape[1]} признаков,
{len(np.unique(y))} классов")

    # Параметры эксперимента
    n_runs = 30
    n_estimators = 20
    max_depth = 5

    print(f"\n2. Параметры эксперимента:")
    print(f"   - Количество запусков: {n_runs}")
    print(f"   - Количество деревьев: {n_estimators}")
    print(f"   - Максимальная глубина: {max_depth}")
    print(f"   - max_features: 'sqrt' ( $\sqrt{{X.shape[1]}}$ ) = {int(np.sqrt(X.shape[1]))}")

    # Запуск эксперимента
    print("\n3. Запуск экспериментов...")
    results = run_experiment(X, y, n_runs=n_runs, n_estimators=n_estimators,

```

```

max_depth=max_depth)

# Расчёт статистики
print("\n4. Расчёт статистики...")
stats = calculate_statistics(results)

# Вывод результатов
print("\n" + "=" * 70)
print("РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ")
print("=" * 70)

print("\n□ Средняя точность ± стандартное отклонение (по 30 запускам):\n")

labels_names = {
    'raw_data': 'Сырые данные (обучение+тест на всех)',
    'random_split': 'Обычное разбиение (80/20)',
    'stratified_split': 'Стратифицированное разбиение (80/20)'
}

for key in results.keys():
    print(f" {labels_names[key]:35}: {stats[key]['mean']:.4f} ± {stats[key]['std']:.4f}")
    print(f" (min: {stats[key]['min']:.4f}, max: {stats[key]
['max']:.4f})")
    print()

# Построение графика
print("\n5. Построение графика...")
plot_results(results, stats)

# ИТОГОВЫЕ ВЫВОДЫ
print("\n" + "=" * 70)
print("ВЫВОДЫ")
print("=" * 70)

raw_mean = stats['raw_data']['mean']
random_mean = stats['random_split']['mean']
strat_mean = stats['stratified_split']['mean']
random_std = stats['random_split']['std']
strat_std = stats['stratified_split']['std']

print(f"""
1. Сырые данные (обучение на всех данных):
  - Точность: {raw_mean:.4f} ± {stats['raw_data']['std']:.4f}
  - Это ЗАВЫШЕННАЯ оценка (модель просто запомнила данные)
  - Не подходит для реальной оценки качества

2. Обычное разбиение (train_test_split):
  - Точность на отложенной выборке: {random_mean:.4f} ± {random_std:.4f}
  - Более честная оценка обобщающей способности
  - Дисперсия: {random_std:.4f}

3. Стратифицированное разбиение (StratifiedShuffleSplit):
  - Точность на отложенной выборке: {strat_mean:.4f} ± {strat_std:.4f}
  - Сохраняет пропорции классов
  - Дисперсия: {strat_std:.4f}
""")

if strat_std < random_std:
    print(f" □ Стратифицированное разбиение дало МЕНЬШУЮ дисперсию:")
    print(f" Уменьшение std на {random_std - strat_std:.4f}")
else:
    print(f" △ Обычное разбиение дало меньшую дисперсию:")

```

```

    print(f"    Разница std: {strat_std - random_std:.4f}")

    if strat_mean > random_mean:
        print(f"\n    □ Стратифицированное разбиение показало ЛУЧШУЮ среднюю
точность:")
        print(f"        +{strat_mean - random_mean:.4f} к accuracy")
    elif random_mean > strat_mean:
        print(f"\n    △ Обычное разбиение показало лучшую точность:")
        print(f"        +{random_mean - strat_mean:.4f} к accuracy")
    else:
        print(f"\n    □ Оба метода показали одинаковую точность")

    print("\n" + "=" * 70)
    print("ЭКСПЕРИМЕНТ ЗАВЕРШЁН")
    print("=" * 70)

if __name__ == "__main__":
    main()

```

реализация с scikit-learn

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import load_iris
from sklearn.ensemble import ExtraTreesClassifier
from sklearn.model_selection import train_test_split, StratifiedShuffleSplit
from sklearn.metrics import accuracy_score
import warnings

warnings.filterwarnings('ignore')

#
=====
#
# ФУНКЦИИ ДЛЯ ЭКСПЕРИМЕНТОВ
#
=====
#

def run_experiment_sklearn(X, y, n_runs=30, n_estimators=20, max_depth=5,
random_state=42):
    """
    Запуск эксперимента с тремя вариациями обучения используя sklearn

    Параметры:
        X: признаки
        y: целевая переменная
        n_runs: количество запусков
        n_estimators: количество деревьев в ансамбле
        max_depth: максимальная глубина деревьев
        random_state: базовое значение для воспроизводимости

    Возвращает:
        results: словарь с результатами ассурасу для каждого подхода
    """

    results = {
        'raw_data': [], # обучение и тест на всех данных

```

```

'random_split': [], # обычное случайное разбиение
'stratified_split': [] # стратифицированное разбиение
}

print("Запуск экспериментов:")
for run in range(n_runs):
    print(f" Итерация {run + 1}/{n_runs}", end='\r')

    # Устанавливаем seed для воспроизводимости
    current_seed = random_state + run if random_state else None

    # ===== 1. Сырые данные (обучение и тест на всех данных) =====
    model_raw = ExtraTreesClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        max_features='sqrt',
        random_state=current_seed,
        n_jobs=-1 # используем все ядра процессора
    )
    model_raw.fit(X, y)
    accuracy_raw = accuracy_score(y, model_raw.predict(X))
    results['raw_data'].append(accuracy_raw)

    # ===== 2. Метод А: Обычное случайное разбиение =====
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=0.2,
        random_state=current_seed,
        shuffle=True # перемешиваем данные
    )

    model_random = ExtraTreesClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        max_features='sqrt',
        random_state=current_seed,
        n_jobs=-1
    )
    model_random.fit(X_train, y_train)
    accuracy_random = accuracy_score(y_test, model_random.predict(X_test))
    results['random_split'].append(accuracy_random)

    # ===== 3. Метод В: Стратифицированное разбиение =====
    sss = StratifiedShuffleSplit(
        n_splits=1, # одно разбиение
        test_size=0.2, # 20% на тест
        random_state=current_seed
    )

    # Получаем единственное разбиение
    train_idx, test_idx = next(sss.split(X, y))
    X_train_strat, X_test_strat = X[train_idx], X[test_idx]
    y_train_strat, y_test_strat = y[train_idx], y[test_idx]

    model_strat = ExtraTreesClassifier(
        n_estimators=n_estimators,
        max_depth=max_depth,
        max_features='sqrt',
        random_state=current_seed,
        n_jobs=-1
    )
    model_strat.fit(X_train_strat, y_train_strat)

```

```

    accuracy_strat = accuracy_score(y_test_strat, model_strat.predict(X_test_strat))
    results['stratified_split'].append(accuracy_strat)

print() # переводим строку после прогресс-бара
return results

def calculate_statistics(results):
    """Вычисляет статистические метрики для каждого подхода"""
    stats = {}
    for key, values in results.items():
        stats[key] = {
            'mean': np.mean(values),
            'std': np.std(values),
            'min': np.min(values),
            'max': np.max(values),
            'median': np.median(values),
            'q1': np.percentile(values, 25), # первый квартиль
            'q3': np.percentile(values, 75) # третий квартиль
        }
    return stats

def print_results_table(stats):
    """Выводит результаты в виде красивой таблицы"""
    print("\n" + "=" * 80)
    print("РЕЗУЛЬТАТЫ ЭКСПЕРИМЕНТОВ".center(80))
    print("=" * 80)

    # Заголовки
    print(f"\n{'Метод':<35} {'Среднее':<12} {'Стд. откл.':<12} {'Min':<10} {'Max':<10}")
    print("-" * 80)

    # Названия методов
    method_names = {
        'raw_data': 'Сырые данные (обучение+тест)',
        'random_split': 'Обычное разбиение (80/20)',
        'stratified_split': 'Стратифицированное разбиение (80/20)'
    }

    for key, name in method_names.items():
        s = stats[key]
        print(f"{'name':<35} {s['mean']:.4f} ± {s['std']:.4f} "
              f"{s['min']:.4f} {s['max']:.4f}")

    print("-" * 80)

def plot_results_comparison(results, stats):
    """Строит улучшенный график сравнения результатов"""

    fig, axes = plt.subplots(1, 2, figsize=(14, 6))

    # 1. Boxplot (слева)
    ax1 = axes[0]
    data_to_plot = [results['raw_data'], results['random_split'], results['stratified_split']]

    labels = [
        f'Сырые данные\n(обучение+тест)',
        f'Обычное разбиение\n(80/20)',
        f'Стратифицированное\nразбиение (80/20)'
    ]

```

```

bp = ax1.boxplot(data_to_plot, labels=labels, patch_artist=True)

# Настройка цветов
colors = ['#FF6B6B', '#4ECDC4', '#45B7D1']
for patch, color in zip(bp['boxes'], colors):
    patch.set_facecolor(color)
    patch.set_alpha(0.7)

# Добавляем средние значения
means = [stats['raw_data']['mean'], stats['random_split']['mean'], stats['stratified_split']
['mean']]
for i, mean in enumerate(means, 1):
    ax1.scatter(i, mean, color='darkred', s=100, zorder=3,
                marker='D', edgecolors='black', linewidth=1.5)
    ax1.annotate(f'{mean:.3f}', xy=(i, mean), xytext=(i + 0.05, mean),
                fontsize=9, fontweight='bold')

ax1.set_ylabel('Accuracy (Точность)', fontsize=12)
ax1.set_title('Распределение точности при разных подходах\n'
              f'({len(results["raw_data"])} запусков)', fontsize=12)
ax1.set_ylim(0.7, 1.05)
ax1.grid(axis='y', alpha=0.3)

# Добавляем аннотацию
ax1.text(0.02, 0.02,
        'Интерпретация:\n'
        '• Сырые данные завышают оценку\n'
        '• Разбиение даёт честную оценку\n'
        '• Стратификация ↓ дисперсию',
        transform=ax1.transAxes, fontsize=9,
        bbox=dict(boxstyle='round', facecolor='wheat', alpha=0.5))

# 2. Линейный график с доверительными интервалами (справа)
ax2 = axes[1]

x_pos = np.arange(3)
means_plot = [stats['raw_data']['mean'], stats['random_split']['mean'],
stats['stratified_split']['mean']]
stds_plot = [stats['raw_data']['std'], stats['random_split']['std'], stats['stratified_split']['std']]

bars = ax2.bar(x_pos, means_plot, yerr=stds_plot, capsize=10,
               color=colors, alpha=0.7, edgecolor='black', linewidth=1.5)

# Добавляем значения на столбцы
for i, (bar, mean, std) in enumerate(zip(bars, means_plot, stds_plot)):
    height = bar.get_height()
    ax2.text(bar.get_x() + bar.get_width() / 2., height + 0.01,
             f'{mean:.3f}\n±{std:.3f}', ha='center', va='bottom', fontsize=9)

ax2.set_xticks(x_pos)
ax2.set_xticklabels(['Сырые данные\n(обучение+тест)',
                    'Обычное\nразбиение',
                    'Стратифицированное\nразбиение'], fontsize=9)
ax2.set_ylabel('Accuracy (Точность)', fontsize=12)
ax2.set_title('Средняя точность ± доверительный интервал (95%)', fontsize=12)
ax2.set_ylim(0.7, 1.05)
ax2.grid(axis='y', alpha=0.3)

plt.tight_layout()
plt.show()

```



```

def analyze_class_distribution(y):
    """Анализирует распределение классов в данных"""
    unique, counts = np.unique(y, return_counts=True)
    print("\n АНАЛИЗ РАСПРЕДЕЛЕНИЯ КЛАССОВ")
    print("=" * 50)
    for cls, count in zip(unique, counts):
        percentage = count / len(y) * 100
        print(f" Класс {cls}: {count} объектов ({percentage:.1f}%)")
    return dict(zip(unique, counts))

def compare_splitting_methods_demo(X, y):
    """Демонстрация разницы между обычным и стратифицированным разбиением"""
    print("\n" + "=" * 80)
    print("ДЕМОНСТРАЦИЯ РАЗНИЦЫ МЕТОДОВ РАЗБИЕНИЯ".center(80))
    print("=" * 80)

    # 5 запусков для демонстрации
    n_demo = 5
    random_props = []
    strat_props = []

    for i in range(n_demo):
        # Обычное разбиение
        _, y_test_rand, _, _ = train_test_split(X, y, test_size=0.2, random_state=i)
        rand_class0_prop = np.sum(y_test_rand == 0) / len(y_test_rand)
        random_props.append(rand_class0_prop)

        # Стратифицированное разбиение
        sss = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=i)
        train_idx, test_idx = next(sss.split(X, y))
        y_test_strat = y[test_idx]
        strat_class0_prop = np.sum(y_test_strat == 0) / len(y_test_strat)
        strat_props.append(strat_class0_prop)

    true_prop = np.sum(y == 0) / len(y)

    print(f"\nИсходная доля класса 0: {true_prop:.3f} ({true_prop * 100:.1f}%)\n")
    print("Запуск | Обычное разбиение | Стратифицированное")
    print("-" * 50)
    for i in range(n_demo):
        print(f" {i + 1:2d} | {random_props[i]:.3f} | {strat_props[i]:.3f}")

    print(f"\nСреднее отклонение от исходной доли:")
    print(f" Обычное разбиение: {np.mean(np.abs(np.array(random_props) -
true_prop)):.4f}")
    print(f" Стратифицированное: {np.mean(np.abs(np.array(strat_props) -
true_prop)):.4f}")

#
=====
#
# ОСНОВНАЯ ПРОГРАММА
#
=====
#

def main():
    print("=" * 80)
    print("ЭКСПЕРИМЕНТ: Сравнение подходов к обучению Extra Tree (scikit-learn)")

```

```

print("=" * 80)

# 1. Загрузка датасета Iris
print("\n1. Загрузка датасета Iris...")
iris = load_iris()
X = iris.data
y = iris.target
feature_names = iris.feature_names
target_names = iris.target_names

print(f"   Загружено: {X.shape[0]} объектов")
print(f"   Признаков: {X.shape[1]} ({', '.join(feature_names)})")
print(f"   Классов: {len(target_names)} ({', '.join(target_names)})")

# 2. Анализ распределения классов
class_dist = analyze_class_distribution(y)

# 3. Демонстрация разницы методов разбиения
compare_splitting_methods_demo(X, y)

# 4. Параметры эксперимента
n_runs = 30
n_estimators = 20
max_depth = 5

print(f"\n2. Параметры эксперимента:")
print(f"   - Количество запусков: {n_runs}")
print(f"   - Количество деревьев: {n_estimators}")
print(f"   - Максимальная глубина: {max_depth}")
print(f"   - max_features: 'sqrt' ( $\sqrt{{X.shape[1]}}$ ) = {int(np.sqrt(X.shape[1]))}")

# 5. Запуск эксперимента
print("\n3. Запуск экспериментов...")
results = run_experiment_sklearn(X, y, n_runs=n_runs,
                                n_estimators=n_estimators,
                                max_depth=max_depth)

# 6. Расчёт статистики
stats = calculate_statistics(results)

# 7. Вывод результатов
print_results_table(stats)

# 8. Детальный анализ результатов
print("\n" + "=" * 80)
print("ДЕТАЛЬНЫЙ АНАЛИЗ".center(80))
print("=" * 80)

print("\n   Статистика по каждому методу:")
for key in stats.keys():
    method_name = {
        'raw_data': 'Сырые данные',
        'random_split': 'Обычное разбиение',
        'stratified_split': 'Стратифицированное разбиение'
    }[key]

    print(f"\n   {method_name}:")
    print(f"       Медиана: {stats[key]['median']:.4f}")
    print(f"       Q1 (25-й перцентиль): {stats[key]['q1']:.4f}")
    print(f"       Q3 (75-й перцентиль): {stats[key]['q3']:.4f}")
    print(f"       IQR (межквартильный размах): {stats[key]['q3'] - stats[key]['q1']:.4f}")

```

```

# 9. Сравнительный анализ
print("\n" + "=" * 80)
print("ВЫВОДЫ".center(80))
print("=" * 80)

raw_mean = stats['raw_data']['mean']
random_mean = stats['random_split']['mean']
strat_mean = stats['stratified_split']['mean']
random_std = stats['random_split']['std']
strat_std = stats['stratified_split']['std']

print(f"""
1. Сырые данные (обучение на всех данных):
  - Точность: {raw_mean:.4f} ± {stats['raw_data']['std']:.4f}
  - ▲ ЗАВЫШЕННАЯ оценка (модель запомнила данные)
  - Не подходит для оценки реального качества

2. Обычное разбиение (train_test_split):
  - Точность на отложенной выборке: {random_mean:.4f} ± {random_std:.4f}
  - Честная оценка обобщающей способности
  - Дисперсия: {random_std:.4f}

3. Стратифицированное разбиение (StratifiedShuffleSplit):
  - Точность на отложенной выборке: {strat_mean:.4f} ± {strat_std:.4f}
  - Сохраняет пропорции классов
  - Дисперсия: {strat_std:.4f}
""")

# Сравнение методов
print("□ СРАВНЕНИЕ МЕТОДОВ:")
print("-" * 50)

if strat_std < random_std:
    improvement = random_std - strat_std
    print(f"□ Стратифицированное разбиение уменьшило дисперсию на {improvement:.4f}")
else:
    print(f"▲ Обычное разбиение дало меньшую дисперсию")

if strat_mean > random_mean:
    improvement = strat_mean - random_mean
    print(f"□ Стратифицированное разбиение повысило точность на {improvement:.4f}")

# Эффект от стратификации
print(f"\n□ Эффект стратификации:")
print(f"  - Улучшение точности: {(strat_mean - random_mean) * 100:.2f}%")
print(f"  - Снижение дисперсии: {(random_std - strat_std) * 100:.2f}%")

# 10. Построение графика
print("\n4. Построение графика...")
plot_results_comparison(results, stats)

print("\n" + "=" * 80)
print("ЭКСПЕРИМЕНТ ЗАВЕРШЁН".center(80))
print("=" * 80)

if __name__ == "__main__":
    main()

```